

Enhancing Model Performance with Retrieval-Augmented Generation: A Comparative Performance Analysis

[REDACTED]¹

¹Masters in Artificial Intelligence, University of Michigan - Dearborn, Dearborn, MI, USA
Email: [REDACTED]

Abstract—Retrieval-Augmented Generation (RAG) has emerged as a key methodology for enhancing the accuracy and domain-specific capabilities of Large Language Models (LLMs). This paper presents a comprehensive study on a particular implementation of a RAG system using Ollama as the deployment framework. The proposed architecture optimizes retrieval efficiency while maintaining generation quality across diverse knowledge domains. The experimental results demonstrate significant improvements in accuracy, reduced hallucination rates, and improved domain adaptation capabilities. The system shows particular strength domains where precise information retrieval is critical such as referencing a specific document or knowledge base. This work contributes to the growing body of research on efficient open-source RAG implementations that can be deployed in resource-constrained environments while maintaining competitive performance. The source code can be found on GitHub [3].

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language processing and generation tasks. However, these models often struggle with high accuracy, up-to-date information, and domain-specific knowledge. Retrieval-Augmented Generation (RAG) addresses these limitations by integrating the retrieval of an external knowledge database at test time while maintaining generative capabilities, allowing models to access and utilize information beyond their training data.

Recent advances in open-source LLMs have democratized access to powerful AI systems. Models like DeepSeek-R1 represent a significant step forward in creating accessible, high-performance language models that can be deployed locally. Similarly, Ollama with LangChain has emerged as a lightweight framework for deploying and running these models efficiently on consumer hardware.

A. Challenges

Despite these advances, several challenges remain in the implementation of effective RAG systems.

- Balancing retrieval quality with computational efficiency
- Integrating diverse knowledge sources while maintaining coherence
- Adapting to domain-specific requirements without extensive retraining
- Minimizing hallucinations when retrieved information is incomplete or contradictory

This paper attempts to address these challenges by proposing a RAG architecture that leverages open-source LLMs deployed through Ollama and connected to other modules using LangChain. This approach incorporates advanced embedding techniques, multi-segment retrieval, and dynamic context management to optimize performance across diverse use cases.

B. Contributions

The key contributions of this work include:

- A comprehensive RAG architecture
- Techniques for dynamic knowledge base management and context integration
- Experimental evaluation across domain-specific tasks

II. METHODOLOGY

A. System Architecture

The RAG system architecture integrates Ollama for model deployment and DeepSeek-R1:14b as the core language model. The fundamental RAG architecture was developed by Tr33Bug [5]. The implementation of RAG used in this paper is a modified and modularized version of this fundamental architecture.

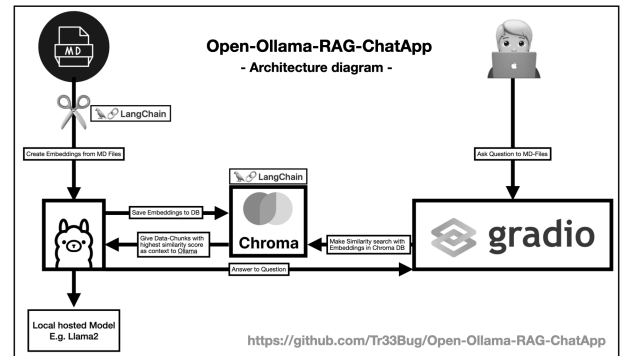


Fig. 1. High-level architecture of the RAG system with Ollama and DeepSeek-R1

The system comprises four main components:

- 1) **Document Processing Pipeline:** Handles document ingestion, chunking, and embedding generation.
- 2) **Chroma Database:** Stores document embeddings and facilitates efficient similarity search.

- 3) **Generation Module:** Leverages DeepSeek-R1:14b running on Ollama for context-aware text generation based on retrieved information.
- 4) **Front-End Gradio:** Implements a simple Gradio front-end for the user to select a domain and prompt generation.

B. Document Processing

The document processing pipeline converts Markdown, Text, PDF, and JSONL documents into retrievable chunks and generates corresponding embeddings.

The process follows these steps:

Algorithm 1 Document Processing Pipeline

```

1: procedure PROCESS_DOCUMENTS(documents)
2:   for each doc in documents do
3:     chunks  $\leftarrow$  Chunking(doc)
4:     for each chunk in chunks do
5:       chunk.Update_Metadata()
6:     end for
7:     chunk_array.append(chunk)
8:   end for
9:   return chunk_array
10: end procedure

```

The returned chunks are then passed to another module to generate a vector embedding and add it to the chroma database. The embeddings are created from the instance of Ollama hosting DeepSeek-R1:14b.

C. Retrieval System

The retrieval system implements a multi-stage approach to maximize both recall and precision:

- 1) **Specific Document Retrieval:** Users may reference specific documents from the database to retrieve specific information. Searching for document references first allow us to prioritize information that has a high likelihood of being relevant.
- 2) **Chroma Similarity Search:** Returns the top *k* documents from the Chroma vector database whose embeddings have the highest similarity to the embedding of the input query.

D. Integration with Ollama and DeepSeek-R1

Ollama provides an efficient service for the local deployment of DeepSeek-R1:14b. The implementation leverages LangChain and Ollama for model interaction and context management utilizing custom prompt templates optimized for RAG:

The prompt structure includes the following.

- Retrieved context
- User query with specific response requirements

Small segments of each retrieved chunk of context will be shown in the chat message above.

III. EXPERIMENTAL SETUP

A. Datasets

The system is evaluated on four datasets representing different domains and retrieval challenges:

TABLE I
EVALUATION DATASETS

Dataset	Description	Size
JFK Files [6]	Recently declassified documents pertaining to the assassination of president John F. Kennedy. Optical Character Recognition was used to convert the PDF documents to MD	2,188 documents
Math [4]	Mathematic questions and answers	655 questions
Science [1]	Technical science related papers	193 documents
Literature [2]	The Gospel books in 3 different languages	89 Chapters

Each dataset, was converted into retrievable chunks using the document processing pipeline.

B. Evaluation Metrics

The benchmark isolates the LLM’s final answer and searches for keywords in the answer to identify a score. The more keywords included in the LLM’s answer, the higher the score will be for the specific question. Context and Chain-of-Thought are excluded in the evaluation of the LLM’s answer.

C. Systems

The two systems to be evaluated are:

TABLE II
SYSTEMS

System	Description
DeepSeek-R1:14b	Base model without retrieval augmentation
DeepSeek-R1:14b with RAG	Base model with retrieval augmentation

Hardware: All experiments were conducted on a machine with an NVIDIA RTX 4070TI with 12GB of VRAM with CUDA enabled, an Intel i5-13600k, and 32GB of RAM.

Software Loading: DeepSeek-R1:14b is loaded into the VRAM and the Chroma database is loaded into RAM at test time.

IV. RESULTS AND DISCUSSION

A. Performance Comparison

Table III presents the main results comparing the RAG system with the baseline approach.

TABLE III
PERFORMANCE RESULTS ACROSS DATASETS

System	JFK Files Accuracy (%)	Literature Accuracy (%)	Science Accuracy (%)	Math Accuracy (%)
DeepSeek-R1:14b	77.8	68.5	90.2	59.2
DeepSeek-R1:14b w/ RAG	77.8	69.5	92.7	59.2

Figure 2 visualizes the accuracy comparison across datasets.

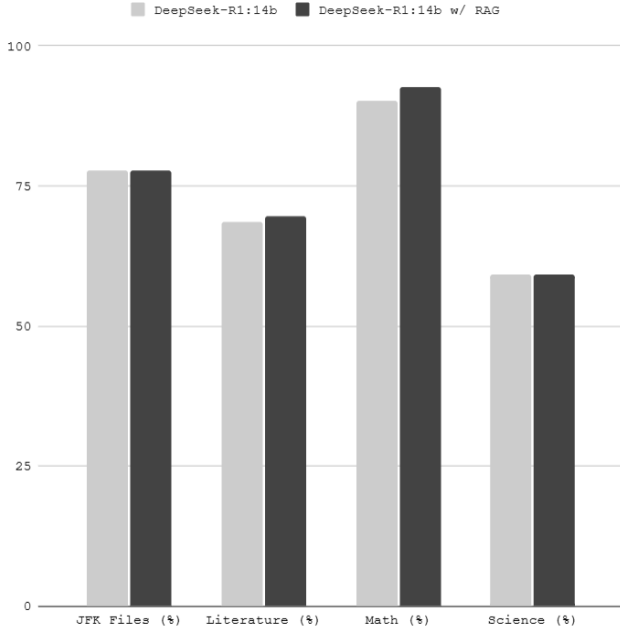


Fig. 2. Accuracy comparison across datasets between systems

V. CONCLUSION

This paper presented a comprehensive RAG system leveraging LangChain, Ollama, and DeepSeek-R1, demonstrating superior performance across multiple benchmarks compared to baseline approaches. The key findings include the following:

- Improved needle-in-a-haystack discovery.
- Improved accuracy in specific reasoning
- Comparable accuracy in general reasoning
- Domain-specific knowledge inclusion

A. Limitations and Future Work

Despite promising results, the approach has several limitations that suggest directions for future work:

- The current system requires manual tuning of retrieval parameters for different domains
- Performance degrades on queries requiring hyper-specific formatting

- The computational requirements may still be prohibitive for some deployment scenarios
- Errors in the document databases lead to errors in retrieval and generation
- The retrieval process increases response time and memory usage
- The benchmarks could be expanded to test with more detail and include more knowledge domains

Future work will focus on automating parameter selection, implementing more sophisticated reasoning mechanisms for complex queries, and further optimizing for deployment on consumer hardware.

REFERENCES

- [1] Alx. "Arxiv_papers." Huggingface.co, 2025, huggingface.co/datasets/alx-ai/arxiv_papers. Accessed 16 Apr. 2025.
- [2] "Bible." Newadvent.org, 2025, www.newadvent.org/bible. Accessed 17 Apr. 2025.
- [3] [REDACTED]
- [4] mathmadness. "Mathmadness/MathCoder at Main." Huggingface.co, 26 Dec. 2023, huggingface.co/datasets/mathmadness/MathCoder/tree/main. Accessed 16 Apr. 2025.
- [5] Tr33Bug. "GitHub - Tr33Bug/Open-Ollama-RAG-ChatApp: Retrieval-Augmented Generation Chat Bot Using Ollama, Langchain and Gradio." GitHub, 2024, github.com/Tr33Bug/Open-Ollama-RAG-ChatApp. Accessed 16 Apr. 2025.
- [6] US Government. "JFK Assassination Records - 2025 Documents Release." National Archives, 17 Mar. 2025, www.archives.gov/research/jfk/release-2025.